23CSE472 – AI & Robotics Lab

Worksheet – 4

Manipulator Representation and Forward Kinematics

1. Representation Method 1 - TRCHAIN

> T = TRCHAIN2(S) is a homogeneous transform (3x3) that results from compounding a number of elementary transformations defined by the string S. The string S comprises a number of tokens of the form X(ARG) where X is one of Tx, Ty, or R. ARG is an arbitrary MATLAB expression that can include constants or workspace variables.
>
> T = TRCHAIN2(S, Q) as above but the expression for ARG can also contain a variable 'qJ'which selects the Jth value from the passed vector Q (1N).
>
> T = TRCHAIN(S) is a homogeneous transform (4x4) that results from compounding a number of elementary transformations defined by the string S. The string S comprises a number of tokens of the form X(ARG) where X is one of Tx, Ty, Tz, Rx, Ry, or Rz. ARG is an arbitrary MATLAB expression that can include constants or workspace variables.
>
> T = TRCHAIN(S, Q) as above but the expression for ARG can also contain a variable 'qJ'which selects the Jth value from the passed vector Q (1N).

a) 2 Joint Planar Manipulator

```
>> a1=1
a1 =
     1

>>a2=1
a2 =
     1

>>q1=0.2
q1 =
    0.2000

>>q2=0.3
q2 =
    0.3000

>>trchain2('R(q1),Tx(a1),R(q2),Tx(a2)',[q1,q2]) %2 Joint
Planar Manipulator
ans =
```

```
    0.8776   -0.4794    1.8576
    0.4794    0.8776    0.6781
         0         0    1.0000
```

b) 2 Joint Planar Manipulator using symbolic variables

```
>> syms q1 q2 a1 a2

>> trchain2('R(q1),Tx(a1),R(q2),Tx(a2)',[q1,q2])
```

c) 3 Joint Planar Manipulator using symbolic variables

```
>> syms q1 q2 q3 a1 a2 a3

>> trchain2('R(q1),Tx(a1),R(q2),Tx(a2), R(q2), Tx(a3)',[q1,q2,
q3])  %3 Joint Planar Manipulator
```

d) 4 Joint 3D Manipulator using symbolic variables

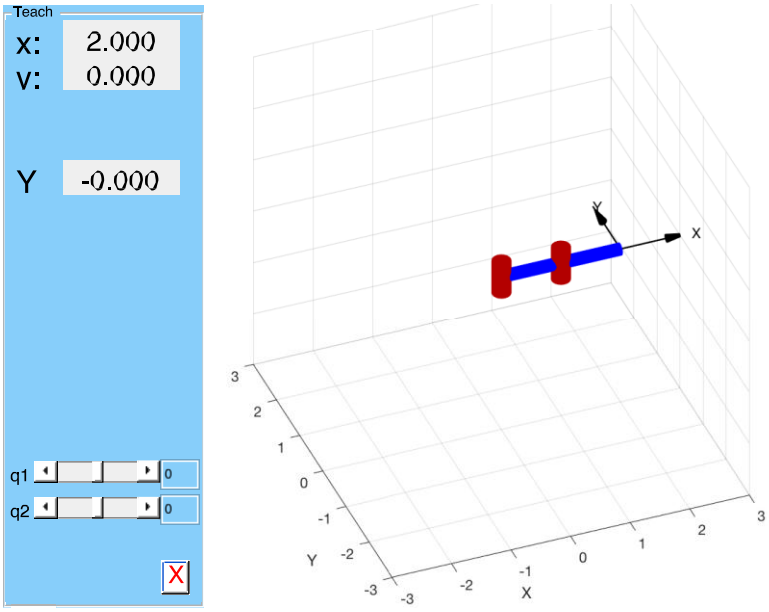
```
syms a1 a2 a3 a4 q1 q2 q3 q4

>> trchain('Rz(q1)Tz(a1)Ry(q2)Tz(a2)Ry(q3)Tz(a3)Ry(q4)Tz(a4)',
[q1 q2 q3 q4] )          %4 Joint Manipulator
```

2. Representation Method 2 - ETS

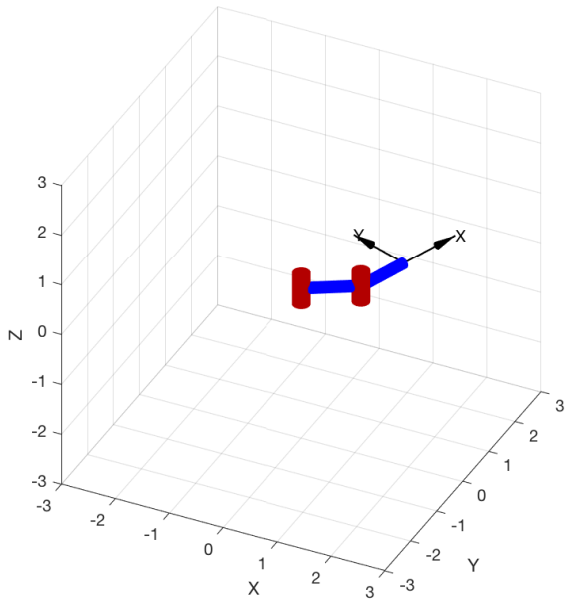
ETS2 - Elementary transform sequence in 2D, this class and package allows experimentation with sequences of spatial transformations in 2D. ETS3 - Elementary transform sequence in 3D, this class and package allows experimentation with sequences of spatial transformations in 3D

a) 2 Joint Planar Manipulator

```
>> import ETS2.*

>> a1=1;a2=1;

>> E = Rz('q1') * Tx(a1) * Rz('q2') * Tx(a2)

E =

Rz(q1)Tx(1)Rz(q2)Tx(1)

>> E.teach

creating new ETS plot
```




```
>> E.fkine( [30 40], 'deg')
ans =
    0.3420   -0.9397    1.208
    0.9397    0.3420     1.44
         0         0         1
>> E.plot( [30, 40], 'deg')
creating new ETS plot
```




```
>> E.structure
ans =
```

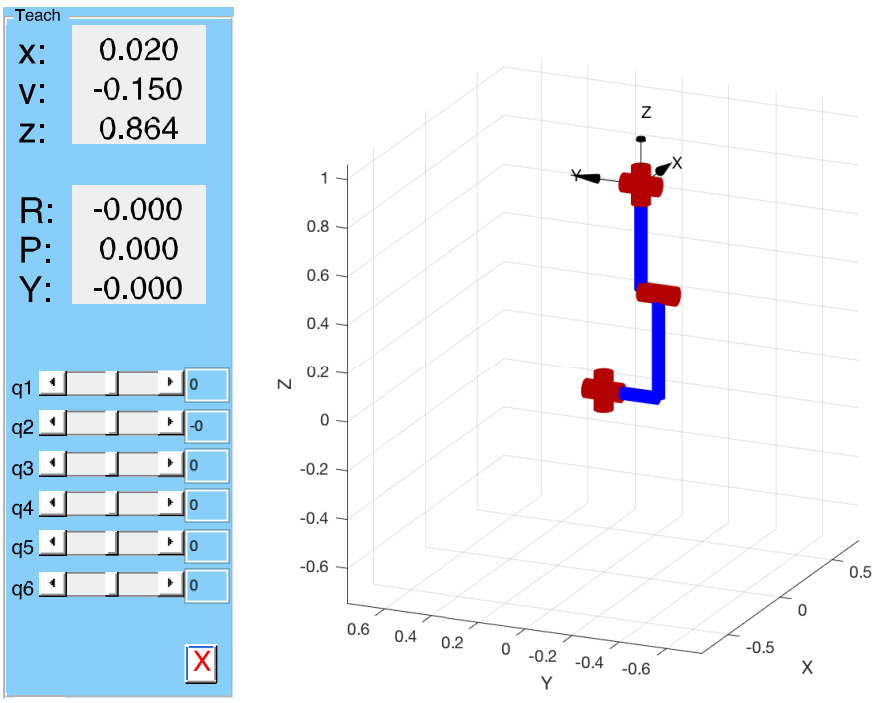
```
    'RR'
```

b) 2 Joint Planar Manipulator with a prismatic joint

```
>> a1=1;

>> E = Rz('q1') * Tx(a1) * Tx('q2')

E =

Rz(q1)Tx(1)Tx(q2)

>> E.structure

ans =

    'RP'
```

c) 3D Manipulator (PUMA 560)

```
>> import ETS3.*

>> L1 = 0; L2 = -0.2337; L3 = 0.4318; L4 = 0.0203; L5 = 0.0837; L6 =
0.4318;

>> E3 = Tz(L1) * Rz('q1') * Ry('q2') * Ty(L2) * Tz(L3) * Ry('q3')*
Tx(L4) * Ty(L5) * Tz(L6) * Rz('q4') * Ry('q5') * Rz('q6');

>> E3.teach
```



3. Representation Method 3 - DH Parameters

Link - A Link object holds all information related to a robot joint and link such as kinematics parameters, rigid-body inertial parameters, motor and transmission parameters.

Examples:

L = Link([0 1.2 0.3 pi/2]);

L = Link('revolute', 'd', 1.2, 'a', 0.3, 'alpha', pi/2);

L = Revolute('d', 1.2, 'a', 0.3, 'alpha', pi/2);

SerialLink Serial-link robot class - A concrete class that represents a serial-link arm-type robot. Each link and joint in the chain is described by a Link-class object using Denavit-Hartenberg parameters (standard or modified).

T = R.fkine(Q, OPTIONS) is the pose of the robot end-effector as an SE3 object for the joint configuration Q (1xN).

If Q is a matrix (KxN) the rows are interpreted as the generalized joint coordinates for a sequence of points along a trajectory. Q(i,j) is the j'th joint parameter for the i'th trajectory point. In this case T is a an array of SE3 objects (K) where the subscript is the index along the path.

R.toradians(Q) converts the joins angles vector from degrees to radians

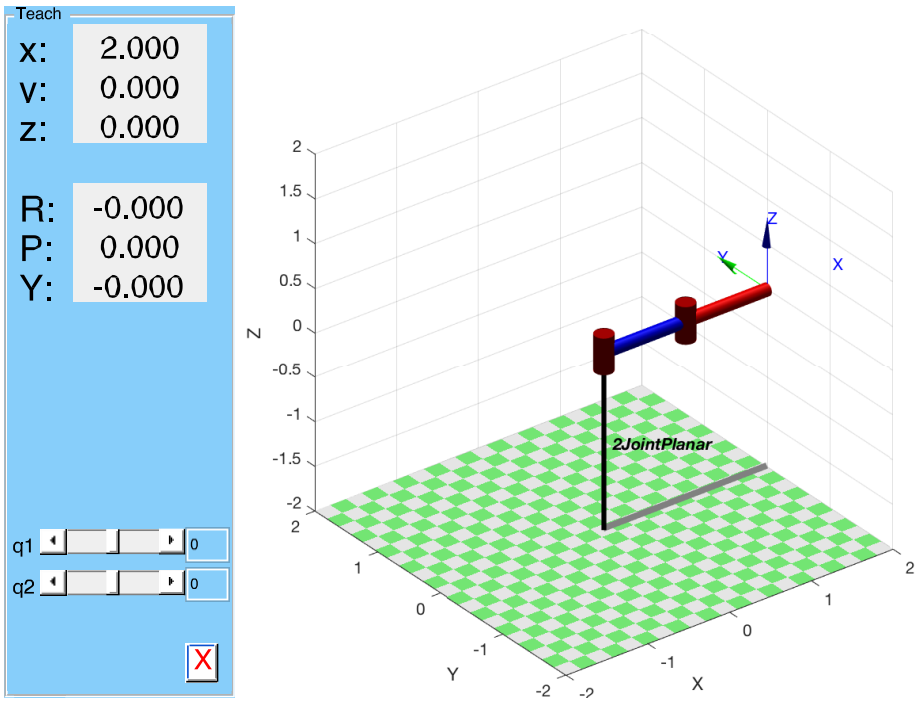
SE3 - Representation of 3D rigid-body motion, this subclasss of RTBPose is an object that represents rigid-body motion in 2D. Internally this is a 33 homogeneous transformation matrix (44) belonging to the group SE(3)

a) 2 Joint Planar Manipulator

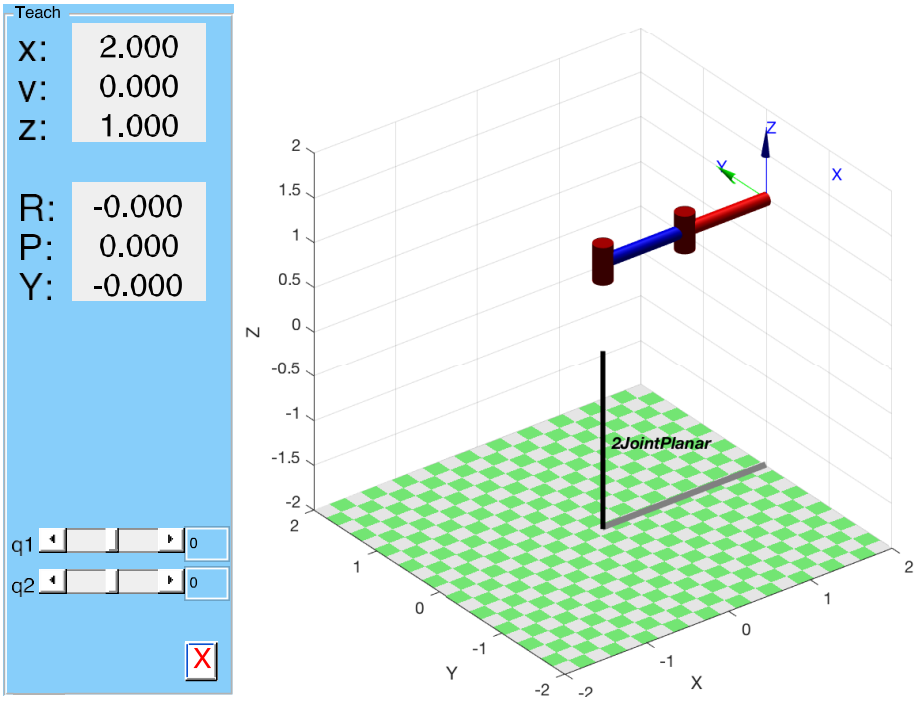
θj	dj	aj	αj
q1	0	a1	0
q2	0	a2	0

```
>> L1 = Link('revolute','a',1)
>> L2 = Link('revolute','a',1)
```

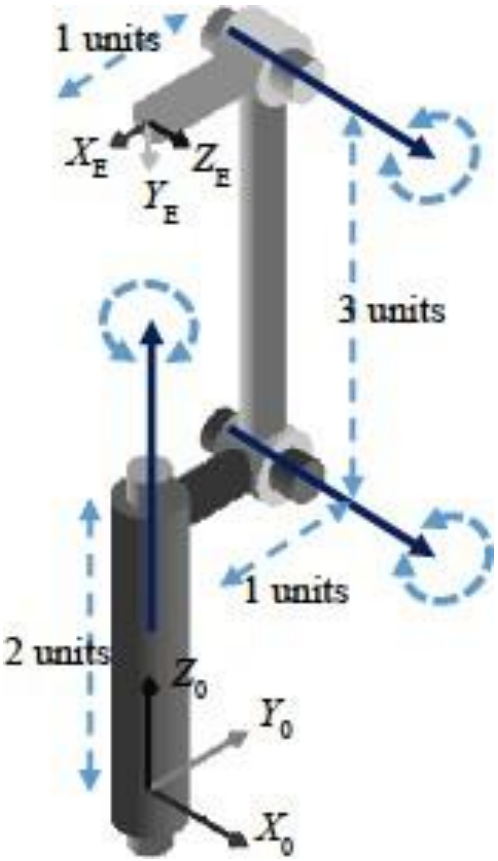
```
>> Robot = SerialLink([L1 L2],'name','2JointPlanar')

Robot =

2JointPlanar:: 2 axis, RR, stdDH, slowRNE
+---+-----------+-----------+-----------+-----------+---------- +
| j |     theta |         d |         a |     alpha |    offset |
+---+-----------+-----------+-----------+-----------+---------- +
|  1|         q1|          0|          1|          0|          0|
|  2|         q2|          0|          1|          0|          0|
+---+-----------+-----------+-----------+-----------+---------- +


>> Robot.fkine(Robot.toradians([30 40]))
ans =
    0.3420   -0.9397         0     1.208
    0.9397    0.3420         0      1.44
         0         0         1         0
         0         0         0         1
>> Robot.teach
```



```
>>Robot.base
ans =
         1         0         0         0
         0         1         0         0
         0         0         1         0
         0         0         0         1

>> Robot.tool
ans =
         1         0         0         0
         0         1         0         0
         0         0         1         0
         0         0         0         1
>> Robot.base = SE3(0,0,1)

Robot =

2JointPlanar:: 2 axis, RR, stdDH, slowRNE
+---+-----------+-----------+-----------+-----------+---------- +
| j |     theta |         d |         a |     alpha |    offset |
+---+-----------+-----------+-----------+-----------+---------- +
|  1|         q1|          0|          1|          0|          0|
|  2|         q2|          0|          1|          0|          0|
+---+-----------+-----------+-----------+-----------+---------- +
base:     t = (0, 0, 1), RPY/xyz = (0, 0, 0) deg
>> Robot.teach
```



b) 3D Manipulator - ARM 1 (As in Lecture slide "Section 3.2")



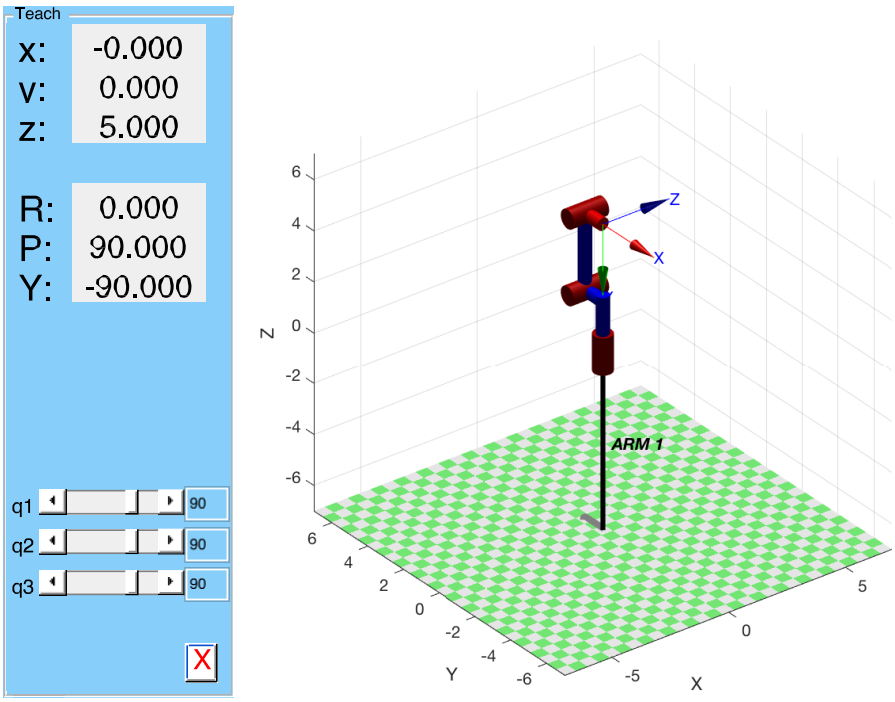
Joint j	θj	dj	aj	αj
1	90	2	1	90
2	90	0	3	0
3	90	0	1	0

Program

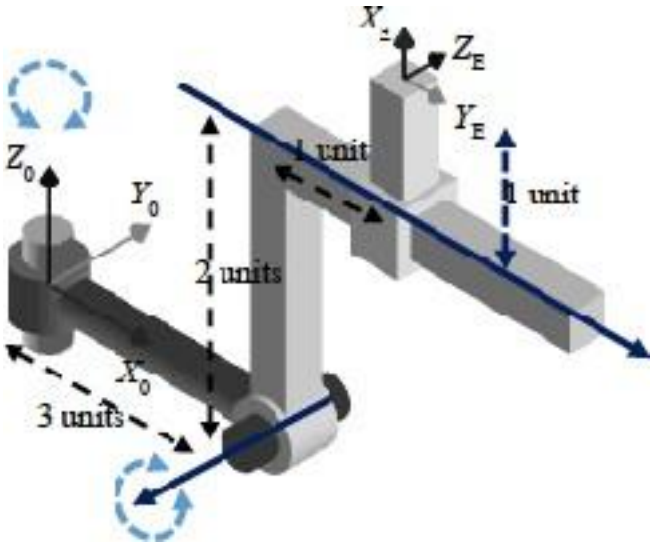
```
% DH parameter representation of Manipulator in Section3.2, ARM 1
clear;
clc;
L1  = Link('revolute','d',2, 'a',1,'alpha',pi/2)
L2  = Link('revolute','d',0, 'a',3,'alpha',0)
L3  = Link('revolute','d',0, 'a',1,'alpha',0)

ARM1 = SerialLink([L1 L2 L3],'name','ARM 1')
ARM1.teach
```

Output

```
ARM1 =

ARM 1:: 3 axis, RRR, stdDH, slowRNE
+---+-----------+-----------+-----------+-----------+---------- +
| j |     theta |         d |         a |     alpha |    offset |
+---+-----------+-----------+-----------+-----------+---------- +
|  1|         q1|          2|          1|     1.5708|          0|
|  2|         q2|          0|          3|          0|          0|
|  3|         q3|          0|          1|          0|          0|
+---+-----------+-----------+-----------+-----------+---------- +
```



c) 3D Manipulator - ARM 2 (As in Lecture slide "Section 3.2")



Joint j	θ_j	d_j	a_j	α_j
1	0	0	3	90
2	90	0	2	90
3	0	1	1	90

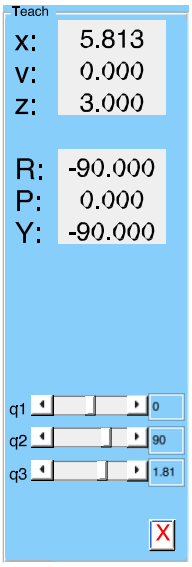
Program

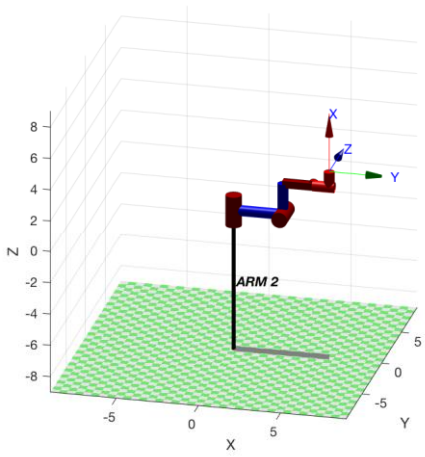
```
% DH parameter representation of Manipulator in Section3.2, ARM 1
clear;
clc;
L1  = Link('revolute','d',0, 'a',3,'alpha',pi/2)
L2  = Link('revolute','d',0, 'a',2,'alpha',pi/2)
L3  = Link('prismatic','theta',0, 'a',1,'alpha',pi/2,'qlim',[0 3],'offset',1)
ARM2 = SerialLink([L1 L2 L3],'name','ARM 2')
ARM2.teach
ARM2.fkine(ARM2.toradians([0 90 0]))
```

Output

```
ARM2 =

ARM 2:: 3 axis, RRP, stdDH, slowRNE

+---+-----------+-----------+-----------+-----------+---------- +
| j |     theta |         d |         a |     alpha |    offset |
+---+-----------+-----------+-----------+-----------+---------- +
|  1|         q1|          0|          3|     1.5708|          0|
|  2|         q2|          0|          2|     1.5708|          0|
|  3|          0|         q3|          1|     1.5708|          1|
+---+-----------+-----------+-----------+-----------+---------- +
```

```
ans =

     0     1     0     4
     0     0     1     0
     1     0     0     3
     0     0     0     1
```





Try Out

1. Represent a articulated 3 joint planar Robot using DH parameters, and Find the pose of the robot in terms of (X,Y and Z) for joint angles (30, 40, 50) in degree.

2. Represent a 3 joint planar Robot using DH parameters, and Find the pose of the robot in terms of (X,Y and Z) for joint angles (15, 45, 55) in degree.